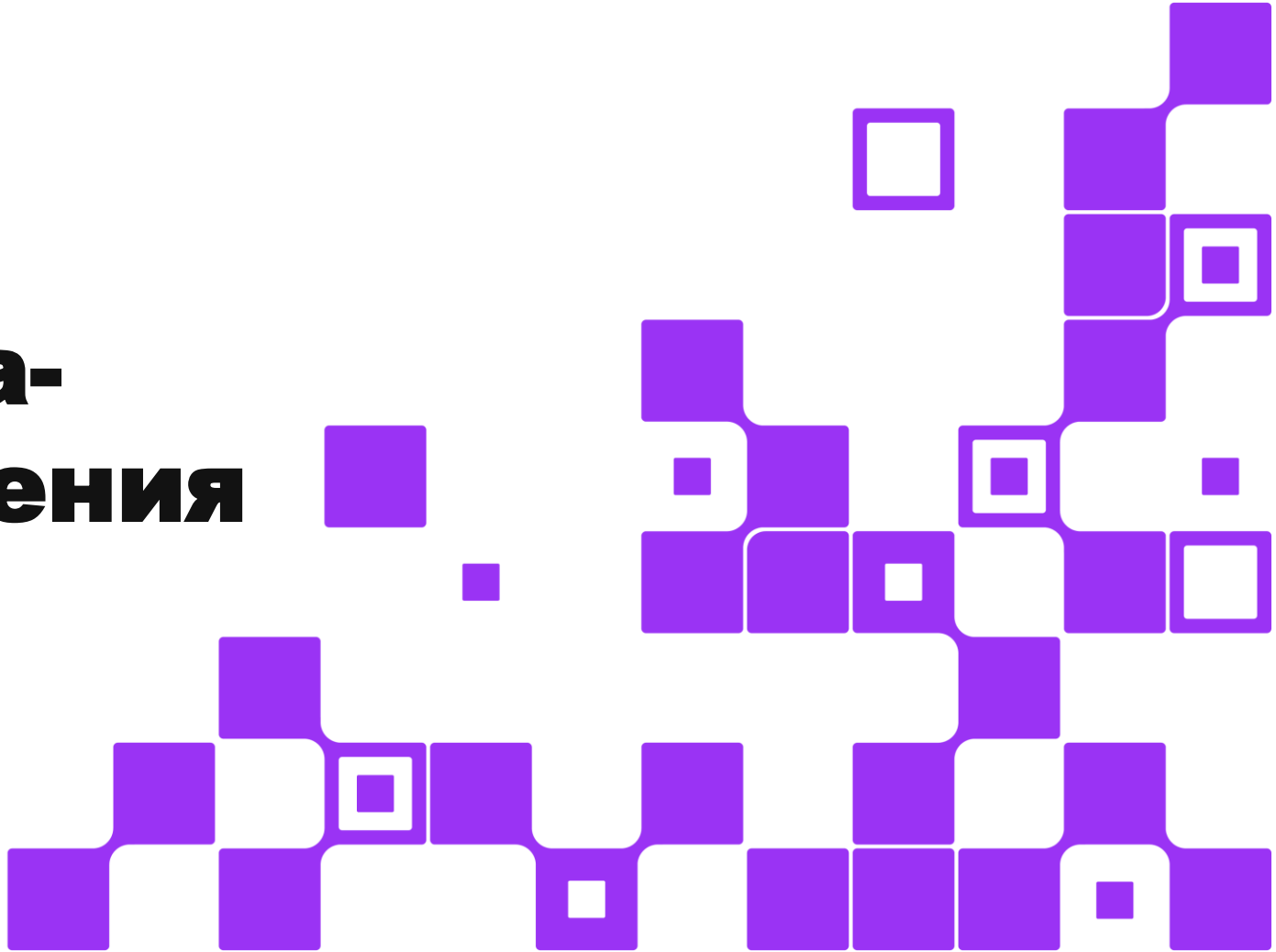
IThub

Лямбда-выражения



Лямбда-выражение представляет собой анонимный (т.е. безымянный) метод. Лямбда-выражения также часто называют замыканиями.

IThub

Он используется для реализации метода, определенного функциональным интерфейсом.

Функциональный интерфейс - это интерфейс, который содержит один и только один абстрактный метод, обычно устанавливающий предполагаемое назначение интерфейса. Используется аннотация @FunctionalInterface

Синтаксис

IThub

новая операция называется лямбда-операцией или операцией стрелки и обозначается с помощью ->

в левой части указываются любые параметры, требующиеся в лямбда-выражении (если параметры не нужны, тогда используется пустой список параметров – ())

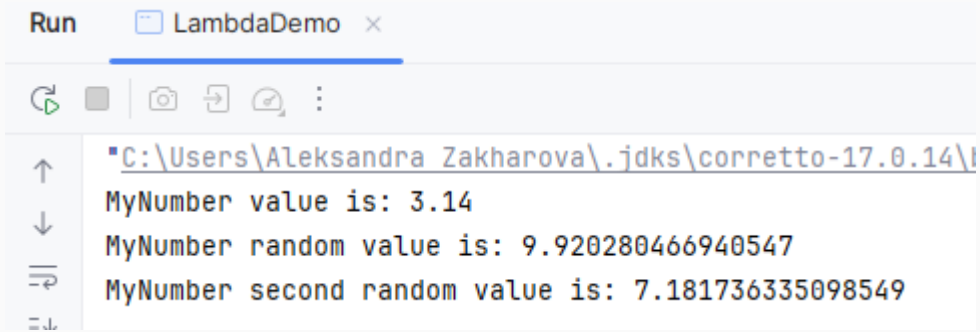
в правой части находится тело лямбда-выражения, которое определяет действия

Пример

IThub

Функциональный интерфейс:

```
interface MyNumber {
    double getNumber();
}
```




```
public class LambdaDemo {

    public static void main(String[] args) {
        MyNumber myNumber = () -> 3.14;
        System.out.println("MyNumber value is: " + myNumber.getNumber());

        myNumber = () -> Math.random() * 10;
        System.out.println("MyNumber random value is: " + myNumber.getNumber());
        System.out.println("MyNumber second random value is: " + myNumber.getNumber());
        System.out.println();
}
```

Функциональный интерфейс:

```
interface NumericTest {
    boolean test(int n);
}
```

```
10 is even
9 is odd

10 is a positive number
-5 is negative number
```

```
public class LambdaDemo {

    public static void main(String[] args) {

        NumericTest isEven = (n) -> (n % 2 == 0);
        System.out.println(isEven.test(10) ? "10 is even" : "10 is odd");
        System.out.println(isEven.test(9) ? "9 is even" : "9 is odd");
        System.out.println();

        NumericTest isNonNeg = (n) -> n >= 0;
        System.out.println(isNonNeg.test(10) ? "10 is a positive number" : "10 is negative number");
        System.out.println(isNonNeg.test(-5) ? "-5 is a positive number" : "-5 is negative number");
        System.out.println();
    }
```

IThub

Функциональный интерфейс:

```
interface NumericTestComparing {
    boolean test(int first, int second);
}
```

```
10 < 100
50 > 5

Process finished with exit code 0
```

```
public class LambdaDemo {

    public static void main(String[] args) {

        NumericTestComparing comparing = (first, second) -> first > second;
        System.out.println(comparing.test(10, 100) ? "10 > 100" : "10 < 100");
        System.out.println(comparing.test(50, 5) ? "50 > 5" : "50 < 5");
    }
```

IThub

интерфейс, ассоциированный с лямбда-выражением, может быть обобщенным (generic)

лямбда-выражение может генерировать исключение.

если инициируется проверяемое исключение, то оно должно быть совместимым с исключением или исключениями, которые перечислены в конструкции throws метода в функциональном интерфейсе

Блочные лямбда-выражения

```
interface StringFunc<T> {
    T func(T t) throws Exception;
}
```

```
public static void main(String[] args) throws Exception {

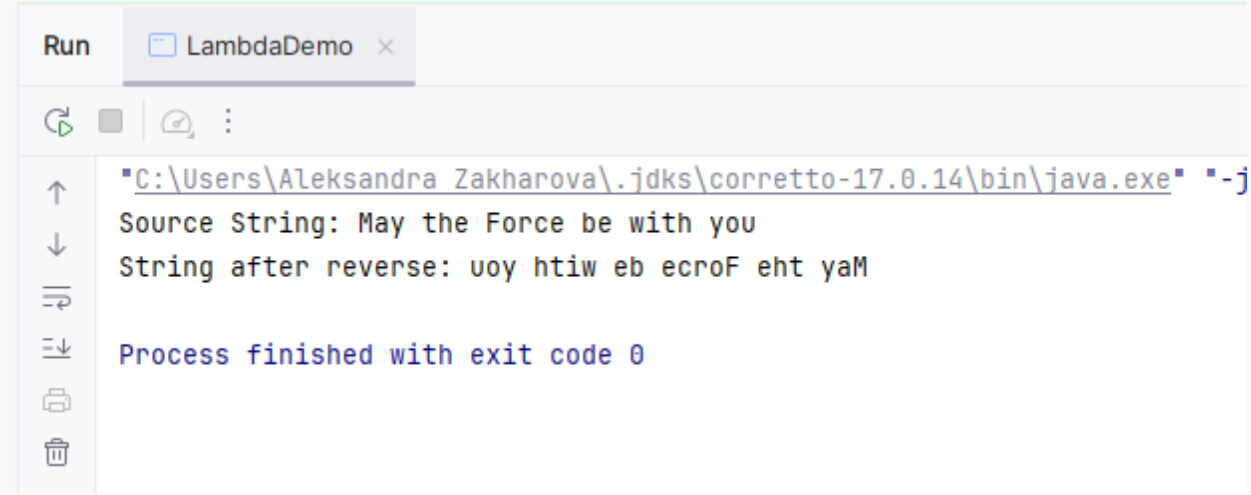
StringFunc<String> reverseString = (str) -> {
        if (str.contains("!")) throw new Exception("Find forbidden character !");

        StringBuilder result = new StringBuilder();
        for (int i = str.length() - 1; i >= 0; i--) {
            result.append(str.charAt(i));
        }
        return result.toString();
    };

    String someString = "May the Force be with you!";
    System.out.println("Source String: " + someString);
    String reversedString = reverseString.func(someString);
    System.out.println("String after reverse: " + reversedString);
}
```

IThub

IThub


Run
LambdaDemo
"C:\Users\Aleksandra Zakharova\.jdks\corretto-17.0.14\bin\java.exe" "-j
Source String: May the Force be with you
String after reverse: uoy htiw eb ecroF eht yaM
Process finished with exit code 0


Run
LambdaDemo
"C:\Users\Aleksandra Zakharova\.jdks\corretto-17.0.14\bin\java.exe" "-javaagent:E:\Programmi
Source String: May the Force be with you!
Exception in thread "main" java.lang.Exception Create breakpoint : Find forbidden character !
at org.example.multithreading.LambdaDemo.lambda$main$5(LambdaDemo.java:49)
at org.example.multithreading.LambdaDemo.main(LambdaDemo.java:61)
Process finished with exit code 1

Ссылки на методы (Method References) появились в Java 8. Они позволяют коротко записывать лямбда-выражения, когда нужно вызвать уже существующий метод или конструктор.
Оператор :: называют оператором ссылки на метод.

IThub

```
List<String> names = List.of("Alex", "Bob", "Charlie");
```

Выведем каждую строку через лямбда-выражение:

```
names.forEach(name -> System.out.println(name));
```

Через ссылку на метод:

```
names.forEach(System.out::println);
```

Предположим, у нас есть список строк:

```
List<String> names = List.of("Alex", "Bob", "Charlie");
```

Выведем каждую строку через лямбда-выражение:

```
names.forEach(name -> System.out.println(name));
```

Через ссылку на метод:

```
names.forEach(System.out::println);
```

Обе записи выполняют одно и то же действие:

- name -> System.out.println(name) — принимаем аргумент name и передаём его методу println.
- System.out::println — передаём ссылку на метод println, а Java сама определяет, какой аргумент ему передать.

Виды ссылок на методы

IThub

1. Ссылка на статический метод	имяКласса::имяСтатическогоМетода
2. Ссылка на метод конкретного объекта	имяОбъекта::имяМетода
3. Ссылка на метод произвольного объекта заданного типа	имяКласса::методЭкземпляра
4. Ссылка на конструктор	имяКласса::new

1. Ссылка на статический метод

```
public class Main {
    public static void main(String[] args) {
        List<String> numbers = List.of("10", "20", "30");

        List<Integer> result = numbers.stream()
                .map(Integer::parseInt)
                .toList();

        System.out.println(result);
    }
}
```

```
[10, 20, 30]
```

Метод parseInt статический,
поэтому перед его именем
указывается класс Integer.

IThub

2. Ссылка на метод конкретного объекта

```
public class Main {
    public static void main(String[] args) {
        String message = "Hello, Java!";

        Supplier<String> supplier = message::toUpperCase;

        System.out.println(supplier.get());
    }
}
```

```
HELLO, JAVA!
```

- message::toUpperCase — ссылка на метод конкретного объекта message.
- функциональный интерфейс Supplier<String> не принимает аргументов, а возвращает строку. При вызове supplier.get() вызывается message.toUpperCase().

IThub

3. Ссылка на метод произвольного объекта заданного типа

```
public class Main {
    public static void main(String[] args) {
        List<String> names = List.of("alex", "bob", "charlie");

        List<String> result = names.stream()
            .map(String::toUpperCase)
            .toList();

        System.out.println(result);
    }
}
```

[ALEX, BOB, CHARLIE]

В отличие от message::toUpperCase, здесь конкретный объект заранее не указан. Каждый элемент потока (stream) становится объектом, для которого вызывается toUpperCase().

4. Ссылки на конструкторы

```
class User {
    private final String name;

    public User(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return name;
    }
}
```

```
List<String> names = List.of("Alex", "Bob", "Charlie");

List<User> users = names.stream()
        .map(User::new)
        .toList();

System.out.println(users);
```

```
[Alex, Bob, Charlie]
```

IThub

Задание

IThub

В компании есть сотрудники:

Employee
String name
int age
double salary

```
@FunctionalInterface
public interface EmployeePredicate {
    boolean test(Employee employee);
}
```

1. Создать список сотрудников (с данными)
2. Создать лямбда-выражение типа Comparator<Employee>, которое сравнивает двух сотрудников по зарплате.
3. Используйте этот компаратор для сортировки списка методом employees.sort(...) по возрастанию зарплаты.
4. Создайте функциональный интерфейс для проверки сотрудника EmployeePredicate.
5. Напишите метод, который принимает список сотрудников и EmployeePredicate, а затем выводит сотрудников, прошедших проверку. Используй обычный цикл, без Stream API

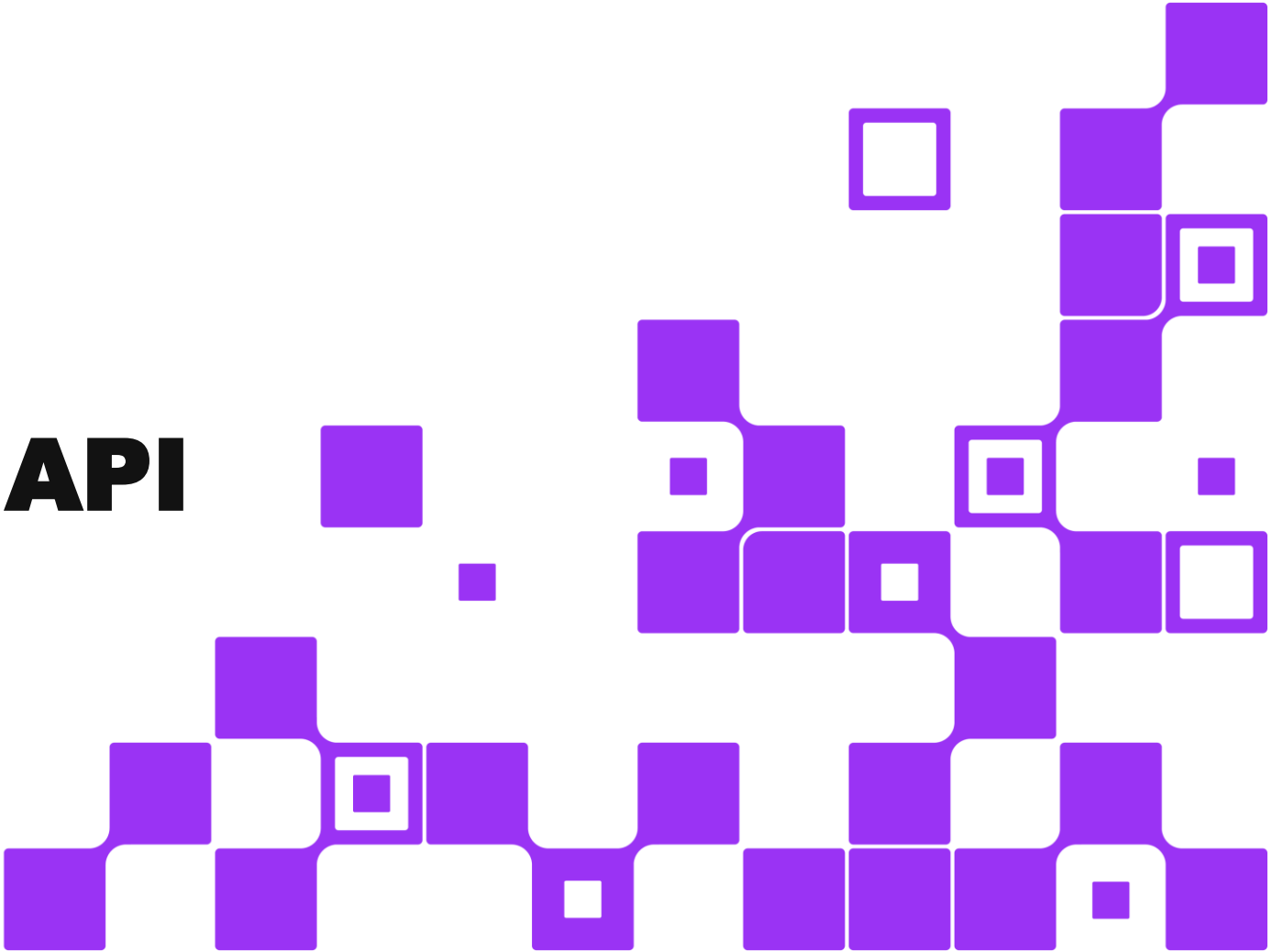
```
Comparator<Employee> salaryComparator =
        (e1, e2) -> Double.compare(
                e1.getSalary(),
                e2.getSalary()
        );
```

```
employees.sort(salaryComparator);
```

```
public static void printEmployees(List<Employee> employees, EmployeePredicate predicate) {
    for (Employee employee : employees) {
        if (predicate.test(employee)) {
            System.out.println(employee.getName());
        }
    }}
```

IThub

Stream API



IThub

Java Stream API предоставляет мощный и лаконичный способ обработки последовательностей элементов, часто из коллекций, в функциональном стиле.

Источник данных: коллекция или другая структура, содержащая элементы

Операции преобразования: фильтрация, сортировка, сопоставление и другие операции над элементами.

Сохранение результата: включает в себя размещение преобразованных данных в новой структуре (например, списке, наборе и т. д.).

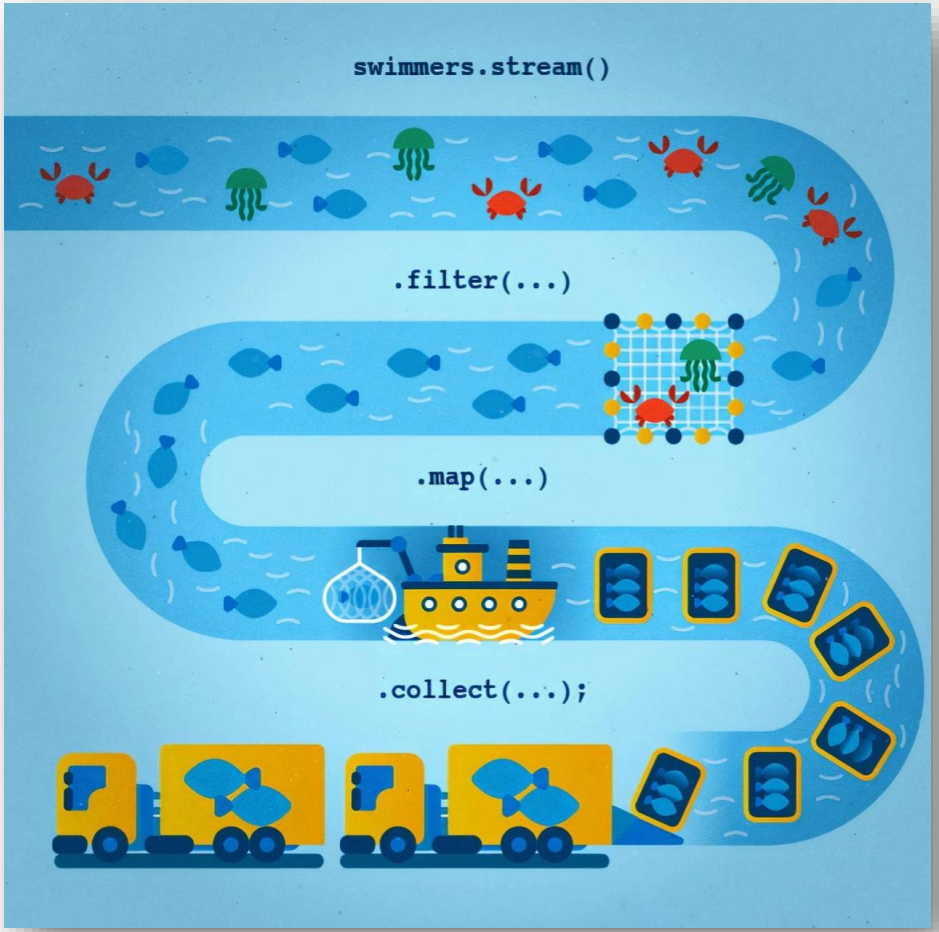
API Stream можно представить как конвейер на рыболовном судне:

Источник данных: река, полная различных морских существ (swimmers), символизирует необработанные данные, которые нужно обработать.

Фильтр: это похоже на рыбаков, которые выбирают из улова только желаемые виды рыбы, исключая нежелательные (filter() метод).

Карта: этот шаг подобен упаковке рыбы в контейнеры. Он преобразует элементы (рыбу) в другую форму данных (с помощью map() метода).

Сбор: на этом этапе вся упакованная рыба загружается в грузовик для дальнейшей транспортировки (collect() операция), которая собирает все результаты в итоговую структуру данных.



Фильтрация (Filtering)

IThub

Используется для удаления значений из коллекции на основе условия.

Для фильтрации элементов коллекции на основе условия используется **метод filter**(). Сохраняются только удовлетворяющие условию элементы.

```
public static List<Integer> filterExample(List<Integer> sourceList) {
    return sourceList.stream()
            .filter(n -> n % 2 == 0)
            .collect(Collectors.toList());
}
```

```
List<Integer> integerList = new ArrayList<>() {{
    add(1);
    add(2);
    add(3);
    add(4);
}};
```

```
@Test
void filterExample() {
    List<Integer> expectedList = new ArrayList<>() {{
        add(2);
        add(4);
    }};

    List<Integer> actualList = StreamApiExamples.filterExample(integerList);
    Assertions.assertEquals(expectedList, actualList);
}
```

Предварительная обработка (Preprocessing)

Используется когда каждое значение в коллекции необходимо изменить на месте.

Метод map() используется для применения функции к каждому элементу коллекции и возврата новой коллекции вычисленных значений.

```
public static List<Integer> preprocessExample(List<Integer> sourceList) {
    return sourceList.stream()
            .map(n -> n*n)
            .collect(Collectors.toList());
}
```

```
List<Integer> integerList =
new ArrayList<>() {{
    add(1);
    add(2);
    add(3);
    add(4);
    add(5);
}};
```

```
@Test
void processExample() {
    List<Integer> expectedList = new ArrayList<>() {{
        add(1);
        add(4);
        add(9);
        add(16);
        add(25);
    }};
    List<Integer> actualList = StreamApiExamples.preprocessExample(integerList);
    Assertions.assertEquals(expectedList, actualList);
}
```

Преобразование (Conversion)

Используется когда необходимо преобразовать коллекцию в другую коллекцию (Collectors.*to…* методы):

- Collectors.toList
- Collectors.toMap
- Collectors.toConcurrentMap
- Collectors.toSet

Преобразование списка строк в карту (Map):

```
public static Map<String, Integer> mapFromList(List<String> sourceList) {
    return sourceList.stream()
            .collect(Collectors.toMap(
                    word -> word,
                    String::length));
}
```

```
List<String> words = new ArrayList<>() {{
    add("name");
    add("Sasha");
    add("apple");
}};
```

```
@Test
void mapFromListExample() {
    Map<String, Integer> expectedMap = new HashMap<>();
    expectedMap.put("name", 4);
    expectedMap.put("Sasha", 5);
    expectedMap.put("apple", 5);

    Map<String, Integer> actualMap = StreamApiExamples.mapFromList(words);
    Assertions.assertEquals(expectedMap, actualMap);
}
```

Преобразование списка (List) в набор (Set) для удаления дубликатов

```
public static Set<String> setFromList(List<String> sourceList) {
    return sourceList.stream()
            .collect(Collectors.toSet());
}
```

```
List<String> words = new ArrayList<>() {{
    add("name");
    add("Sasha");
    add("apple");
    add("Sasha");
}};
```

```
@Test
void setFromListExample() {
    Set<String> expectedSet = new HashSet<>() {{
        add("name");
        add("Sasha");
        add("apple");
    }};

    Set<String> actualSet = StreamApiExamples.setFromList(words);
    Assertions.assertEquals(expectedSet, actualSet);
}
```

Преобразование карты (Map) в список (List)

```
public static List<String> mapToList(Map<String, Integer> map) {
    return map.entrySet().stream()
            .map(e -> e.getKey())
            .collect(Collectors.toList());
}
```

```
Map<String, Integer> map = new HashMap<>() {{
    put("Sasha", 5);
    put("apple", 5);
    put("name", 4);
}};
```

```
@Test
void mapToListExample() {
    List<String> actualList = StreamApiExamples.mapToList(map);
    Assertions.assertTrue(words.containsAll(actualList));
}
```

```
List<String> words = new ArrayList<>() {{
    add("name");
    add("Sasha");
    add("apple");
}};
```

IThub

Сокращение (Reduction)

IThub

Сокращает Collection до одного значения.

Метод reduce() используется для применения функции к каждому элементу коллекции и возврата одного значения.

```
public int reduce(List<Integer> list) {
    return list.stream().reduce(0, Integer::sum);
}
```

```
List<Integer> integerList = new ArrayList<>() {{
    add(1);
    add(2);
    add(3);
    add(4);
    add(5);
}};
```

```
@Test
void reduceExample() {
    int expected = 15;
    int actual = StreamApiExamples.reduce(integerList);
    Assertions.assertEquals(expected, actual);
}
```

Группировка (Grouping)

IThub

Группирует элементы Collection по заданному условию.

Для группировки элементов Collection по условию используется метод **Collectors.groupingBy()**.

```
public class Box {

    private double width;
    private double height;
    private double depth;

    ….

    public double getVolume() {
        return width * height * depth;
    }
    ….
}
```

```
Box box1 = new Box(2,2,4);
Box box2 = new Box(1,2,8);
Box box3 = new Box(1,1,1);

List<Box> boxes = new ArrayList<>() {{
    add(box1);
    add(box2);
    add(box3);
}};
```

```
public static Map<Double, List<Box>> filterExample(List<Box> boxes) {
    return boxes.stream()
            .collect(Collectors.groupingBy(box -> box.getVolume()));
}
```

```
@Test
void groupingByExampleTest() {
    Map<Double, List<Box>> expextedMap = new HashMap<>() {{
        put(16.0, Arrays.asList(box1, box2));
        put(1.0, Collections.singletonList(box3));
    }};
    Map<Double, List<Box>> actualMap = StreamApiExamples.groupingByExample(boxes);
    Assertions.assertEquals(expextedMap, actualMap);
}
```

Поиск (Finding)

IThub

Ищет элемент коллекции, соответствующий условию.

Для поиска используются методы **findFirst()** и **findAny()**:

- findFirst() полезен, когда важно получить первый элемент (например, в упорядоченном потоке), обычно в сочетании с фильтрацией
- findAny() полезен, когда порядок элементов не имеет значения, а необходимо получить результат как можно быстрее

max() и **min()** используются для возврата максимального/минимального значения элемента после обработки потока.

все эти методы возвращают Optional<T>

Класс Optional

IThub

Объект-контейнер, который может содержать или не содержать ненулевое значение.

если значение присутствует, метод isPresent() вернет true и метод get() вернет это значение

метод ifPresent() - выполнение блока кода, если значение присутствует

метод orElse() - возврат значения по умолчанию, если значение отсутствует

```
@Test
void optionalExampleTest() {
    Optional<String> empty = Optional.empty();
    Assertions.assertFalse(empty.isPresent());
```

```
String name = "Sasha";
Optional<String> opt = Optional.of(name);
Assertions.assertTrue(opt.isPresent());
```

аргумент, переданный методу of(), не может быть null (будет NullPointerException)

```
Assertions.assertThrows(NullPointerException.class, () -> {
    Optional.of(null);
});
```

если ожидаем некоторые значения null , мы можем использовать метод ofNullable()

```
String nullValue = null;
Optional<String> optNullable = Optional.ofNullable(nullValue);
Assertions.assertFalse(optNullable.isPresent());
```

- **isPresent()** возвращает true, если упакованное значение **не равно** null
- **isEmpty()** возвращает true, если упакованное значение **равно** null
- метод **orElse** возвращает обёрнутое значение, если оно присутствует, и его аргумент в противном случае

```
String nullValue = null;
String orElseName = Optional.ofNullable(nullValue).orElse("Sasha");
Assertions.assertEquals("Sasha", orElseName);
```

- метод **orElseGet** принимает функциональный интерфейс и возвращает значение вызова, если обёрнутое значение null

```
String nullValue = null;
orElseName = Optional.ofNullable(nullValue).orElseGet(() -> "Sasha");
Assertions.assertEquals("Sasha", name);
```

- метод **orElseThrow** вместо возврата значения по умолчанию при отсутствии упакованного значения выдает исключение

```
Optional.ofNullable(nullValue).orElseThrow(IllegalArgumentException::new);
```

В Java 10 появилась версия метода orElseThrow() без аргументов (выдаёт исключение NoSuchElementException)

Optional VS null

IThub

```
public String someMethod() {
    if (некоторое_условие) {
        return некоторое_значение;
    }
    // в остальных случаях возвращаем null
    return null;
}


// метод где-то вызывается
String variable = someMethod();
if (variable == null) {
    // некоторые действия
} else {
    // некоторые действия
}
```

```
public Optional<String> someMethod() {
    if (некоторое_условие) {
        return Optional.of(некоторое_значение);
    }
    // в остальных случаях возвращаем пустой Optional
    return Optional.empty();
}


// метод где-то вызывается
Optional<String> variableOpt = someMethod();
if(variableOpt.isPresent()) {
    String variable = variableOpt.get();
    // некоторые действия
} else {
    // некоторые действия
}
```

- Если someMethod() объявлен в какой-то библиотеке (api) и нет никакой документации, программист может не заходя в метод использовать его, не зная, что метод может вернуть null, а не строку со значением (не добавит проверки == null) и в его коде может быть NullPointerException
- Зная, что метод возвращает Optional, программист обязательно проверит его на isPresent или isEmpty

- Optional предназначен для использования **в качестве возвращаемого типа**. Использовать его в качестве типа поля не рекомендуется.
- Кроме того, использование Optional в сериализуемом классе приведёт к исключению NotSerializableException

Поиск (Finding)

IThub

```
Box box1 = new Box(2,2,4);
Box box2 = new Box(1,2,8);
Box box3 = new Box(1,1,1);

List<Box> boxes = new ArrayList<>() {{
    add(box1);
    add(box2);
    add(box3);
}};
```

```
public static Optional<Box> findExample(List<Box> boxes) {
    return boxes.stream()
            .filter(box -> box.getVolume() > 5)
            .findFirst();
}
```

```
@Test
void findTest() {
    Optional<Box> actualOpt = StreamApiExamples.findExample(boxes);
    Assertions.assertTrue(actualOpt.isPresent());
    Box box = actualOpt.get();
    Assertions.assertEquals(box, box1);
}
```

Поиск (Finding)

IThub

```
Box box1 = new Box(2,2,4);
Box box2 = new Box(1,2,8);
Box box3 = new Box(1,1,1);

List<Box> boxes = new ArrayList<>() {{
    add(box1);
    add(box2);
    add(box3);
}};
```

```
public static Optional<Box> findAnyExample(List<Box> boxes) {
    return boxes.stream()
            .filter(box -> box.getHeight() == 1)
            .findAny();
}
```

```
@Test
void findAnyTest() {
    Optional<Box> actualOpt = StreamApiExamples.findAnyExample(boxes);
    Assertions.assertTrue(actualOpt.isPresent());
    Box box = actualOpt.get();
    Assertions.assertEquals(box, box3);
}
```

max и min

IThub

```
Box box1 = new Box(2,2,4);
Box box2 = new Box(1,2,8);
Box box3 = new Box(1,1,1);

List<Box> boxes = new ArrayList<>() {{
    add(box1);
    add(box2);
    add(box3);
}};
```

```
@Test
public void findMaxTest() {
    Optional<Box> actualOpt = boxes.stream()
            .filter(box -> box.getVolume() >= 16)
            .max(Comparator.comparing(Box::getWidth));
    Assertions.assertTrue(actualOpt.isPresent());
    Box box = actualOpt.get();
    Assertions.assertEquals(box, box1);
}
```

```
@Test
public void findMinTest() {
    Optional<Box> actualOpt = boxes.stream()
            .filter(box -> box.getVolume() >= 16)
            .min(Comparator.comparing(Box::getWidth));
    Assertions.assertTrue(actualOpt.isPresent());
    Box box = actualOpt.get();
    Assertions.assertEquals(box, box2);
}
```

Сортировка (Sorting)

IThub

Сортирует элементы коллекции.

Для сортировки используется метод **sorted()**.

Есть два варианта метода – без параметров и с параметров Comparator.

Если метод вызван без параметров, возвращает поток, состоящий из элементов этого потока, отсортированных в **естественном порядке** (числа от меньшего к большему, строки в алфавитным порядке)

Сортировка (Sorting)

IThub

```
public static List<String> sortExample(List<String> sourceList) {
    return sourceList.stream()
            .sorted(Comparator.comparingInt(String::length))
            .collect(Collectors.toList());
}
```

```
@Test
void sortTest() {
    List<String> sourseList = new ArrayList<>();
    sourseList.add("Yesterday");
    sourseList.add("Sasha");
    sourseList.add("I");
    sourseList.add("Love");

    List<String> actualList = StreamApiExamples.sortExample(sourseList);
    Assertions.assertEquals("I", actualList.get(0));
    Assertions.assertEquals("Love", actualList.get(1));
    Assertions.assertEquals("Sasha", actualList.get(2));
    Assertions.assertEquals("Yesterday", actualList.get(3));
}
```

distinct() убирает дубликаты из потока и возвращает новый поток

limit() сохраняет указанное количество элементов в начале коллекции и возвращает новый поток

```
List<String> sourseList = new ArrayList<>();
sourseList.add("Yesterday");
sourseList.add("Wednesday");
sourseList.add("Sunday");
sourseList.add("Sasha");
sourseList.add("I");
sourseList.add("Sasha");

List<String> filteringList = sourseList.stream()
        .filter(s -> s.length() >= 5)
        .distinct()
        .sorted()
        .skip(1)
        .limit(2)
        .collect(Collectors.toList());

Assertions.assertTrue(filteringList.size() == 2);
Assertions.assertTrue(filteringList.contains("Sunday"));
Assertions.assertTrue(filteringList.contains("Wednesday"));
```

// [Yesterday, Wednesday, Sunday, Sasha, Sasha]
// [Yesterday, Wednesday, Sunday, Sasha]
// [Sasha, Sunday, Wednesday, Yesterday]
// [Sunday, Wednesday, Yesterday]
// [Sunday, Wednesday]

IThub

Разделение (Partitioning)

IThub

Разделяет элементы колллекции по заданному условию.

Для разделения элементов используется метод **Collectors.partitioningBy().**

Разделение похоже на группировку, за исключением того, что оно возвращает две коллекции — одну для элементов, соответствующих условию, и одну для элементов, не соответствующих условию.

Разделение (Partitioning)

IThub

```
public static Map<Boolean, List<Integer>> partitioningExample(List<Integer> sourceList) {
    return sourceList.stream()
            .collect(Collectors.partitioningBy(n -> n % 2 == 0));
}
```

```
@Test
void partitionTest() {
    List<Integer> evenExpected = new ArrayList<>() {{ add(2); add(4); }};
    List<Integer> oddExpected = new ArrayList<>() {{ add(1);  add(3); add(5); }};

    Map<Boolean, List<Integer>> actualMap = StreamApiExamples.partitioningExample(integerList);
    List<Integer> evenNumbersActual = actualMap.get(true);
    List<Integer> oddNumbersActual = actualMap.get(false);

    Assertions.assertTrue(evenExpected.containsAll(evenNumbersActual));
    Assertions.assertTrue(oddExpected.containsAll(oddNumbersActual));
}
```

Подсчет (Counting)

IThub

Подсчитывает количество элементов, соответствующих условию.

Для подсчета количества элементов, соответствующих условию, используется метод **count().**

```
List<Integer> integerList =
   new ArrayList<>() {{
    add(1);
    add(2);
    add(3);
    add(4);
    add(5);
}};
```

```
public static long countExample(List<Integer> sourceList) {
    return sourceList.stream()
            .filter(n -> n % 2 == 0)
            .count();
}
```

```
@Test
void countTest() {
    long expected = 2;
    long actual = StreamApiExamples.countExample(integerList);
    Assertions.assertEquals(expected, actual);
}
```

Диапазон (Range)

IThub

Создает диапазон значений.

Для создания диапазона значений используется метод **range().**

Существуют специальные классы для создания потоков определенных типов — IntStream, LongStream, DoubleStream и Stream. Эти классы полезны при работе с примитивными числовыми типами. Для преобразования массива в поток используется Arrays.stream().

```
// создаем массив чисел от 1 до 9
int[] numbers = IntStream.range(1, 10).toArray();
```

```
// создаем массив чисел от 1 до 10
int[] numbers = IntStream.rangeClosed(1, 10).toArray();
```

Соответствие (Matching)

Сопоставляет элементы коллекции с предикатом (условием).

Для сопоставления элементов коллекции с предикатом и возврата логического значения используются такие методы, как anyMatch(), allMatch(), и noneMatch().

```
List<String> words = new ArrayList<>() {{
    add("name");
    add("Sasha");
    add("apple");
}};
```

```
@Test
void matchTest() {
    boolean allMatch = words.stream()
            .allMatch(word -> word.length() > 3);
    Assertions.assertTrue(allMatch);

    allMatch = words.stream()
            .allMatch(word -> word.length() > 3000);
    Assertions.assertFalse(allMatch);


    boolean anyMatch = words.stream()
            .anyMatch(word -> word.length() == 4);
    Assertions.assertTrue(anyMatch);

    anyMatch = words.stream()
            .anyMatch(word -> word.length() > 3000);
    Assertions.assertFalse(anyMatch);


    boolean noneMatch = words.stream()
            .noneMatch(word -> word.length() > 3000);
    Assertions.assertTrue(noneMatch);
}
```

IThub

Присоединение (Joining)

Объединяет элементы коллекции в строку.

Для объединения элементов коллекции в строку используется метод **Collectors.joining()**.

```
List<String> words = new ArrayList<>() {{
    add("name");
    add("Sasha");
    add("apple");
}};
```

```
@Test
void joinTest() {
   String joinString = words.stream().collect(Collectors.joining(" - "));
   String expected = "name - Sasha - apple";
   Assertions.assertEquals(expected, joinString);
}
```

IThub

- Задача: найти все коробки, у которых высота (height) больше 1 и объем = 16 в отсортированном порядке по ширине (width)

```
List<Box> boxList = new ArrayList<>();
   boxList.add(new Box(1, 2, 8));    // vol=16
   boxList.add(new Box(5, 5, 5));
   boxList.add(new Box(2, 2, 4));    // vol=16
   boxList.add(new Box(1, 16, 1));  // vol=16
   boxList.add(new Box(6, 6, 6));
```

Через цикл

```
   List<Box> foundBoxes = new ArrayList<>();
   for (Box box : boxList) {
       if (box.getHeight() > 1 && box.getVolume() == 16)
           foundBoxes.add(box);
   }
   foundBoxes.sort(Comparator.comparing(Box::getWidth));
```

```
public class Box {

    private double width;
    private double height;
    private double depth;

    ….

    public double getVolume() {
        return width * height * depth;
    }
    ….
}
```

Через stream api

```
List<Box> foundBoxesStream = boxList.stream()
        .filter(box -> box.getHeight() > 1 && box.getVolume() == 16)
        .sorted(Comparator.comparingDouble(Box::getWidth))
        .collect(Collectors.toList());
```

IThub

Метод **peek()** позволяет добавить в поток промежуточную операцию, которая выполняет действие над каждым элементом, не изменяя сам поток. Это полезно для отладки или логирования.

```
List<String> sourseList = new ArrayList<>();
sourseList.add("Yesterday");
sourseList.add("Wednesday");
sourseList.add("Sunday");
sourseList.add("Sasha");
sourseList.add("I");
sourseList.add("Sasha");

List<String> filteringList = sourseList.stream()
        .filter(s -> s.length() >= 5)
        .peek(e -> System.out.println("Filtered value: " + e))
        .distinct()
        .peek(e -> System.out.println("distinct value: " + e))
        .sorted()
        .peek(e -> System.out.println("sorted value: " + e))
        .skip(1)
        .limit(2)
        .collect(Collectors.toList());
```

```
Filtered value: Yesterday
distinct value: Yesterday
Filtered value: Wednesday
distinct value: Wednesday
Filtered value: Sunday
distinct value: Sunday
Filtered value: Sasha
distinct value: Sasha
Filtered value: Sasha
sorted value: Sasha
sorted value: Sunday
sorted value: Wednesday

Process finished with exit code 0
```

IThub

Как работает поток

Stream API в Java обрабатывает данные через цепочки промежуточных и конечных операций, но важной особенностью является то, что **каждый элемент** потока **проходит весь конвейер** шаг за шагом.

Операции не применяются к коллекции в целом, а последовательно работают с каждым ее элементом.

Такой пошаговый подход упрощает и делает параллельную обработку более безопасной. Поскольку каждый элемент обрабатывается независимо, переход от последовательного потока к параллельному легко осуществляется.

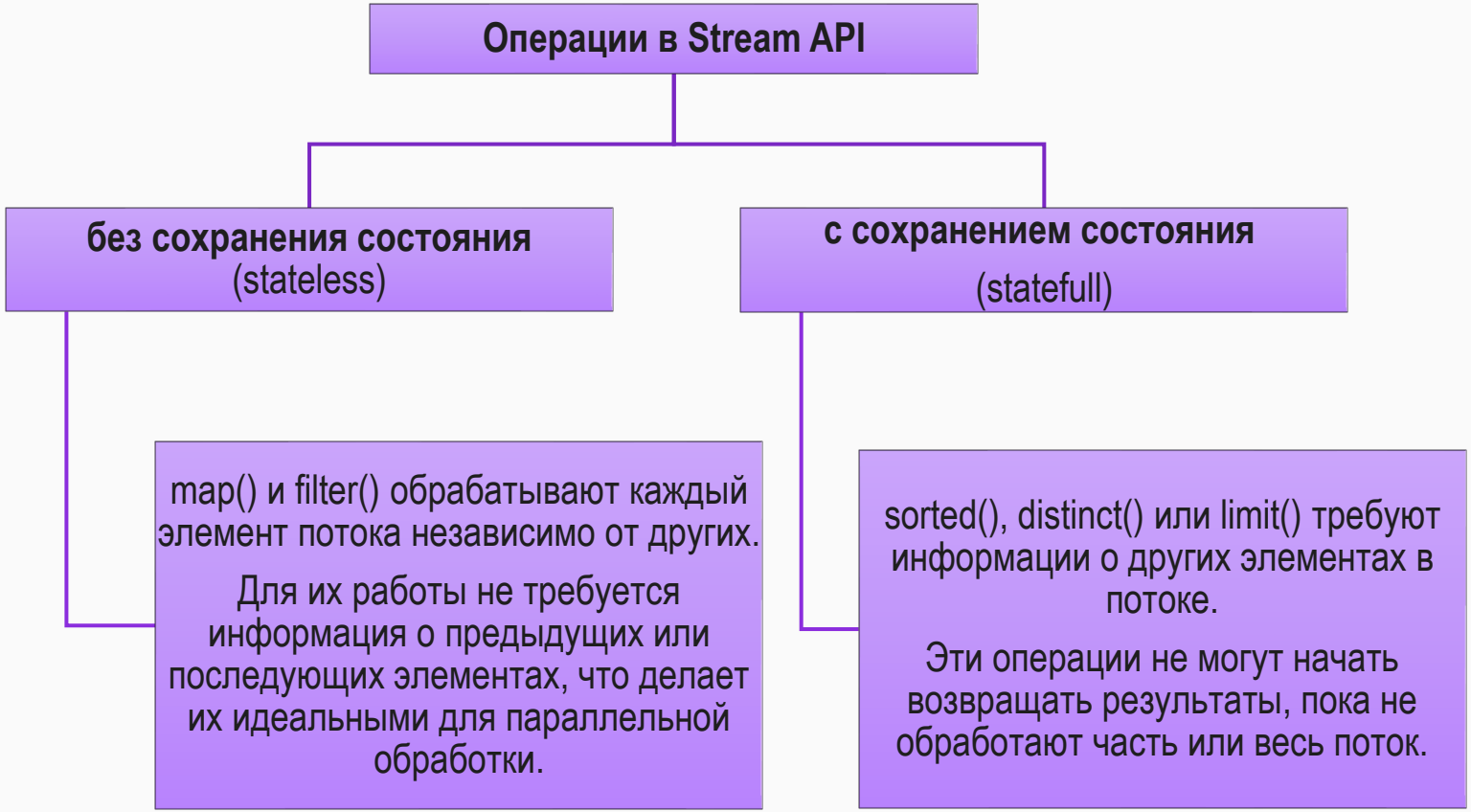
```
final List<String> list = List.of("one", "two", "three");

list.stream()
        .filter(s -> {
            System.out.println("filter: " + s);
            return s.length() <= 3;
        })
        .map(s1 -> {
            System.out.println("map: " + s1);
            return s1.toUpperCase();
        })
        .forEach(x -> {
            System.out.println("forEach: " + x);
        });
```

```
filter: one
map: one
forEach: ONE
filter: two
map: two
forEach: TWO
filter: three

Process finished with exit code 0
```

IThub


Операции в Stream API
без сохранения состояния
(stateless)
с сохранением состояния
(statefull)
map() и filter() обрабатывают каждый элемент потока независимо от других.
Для их работы не требуется информация о предыдущих или последующих элементах, что делает их идеальными для параллельной обработки.
sorted(), distinct() или limit() требуют информации о других элементах в потоке.
Эти операции не могут начать возвращать результаты, пока не обработают часть или весь поток.

IThub

Если конвейер состоит только из операций без сохранения состояния, поток может быть обработан «за один проход», что делает выполнение быстрым и эффективным.

Однако при добавлении операций с отслеживанием состояния поток разделяется на разделы, и каждый раздел должен завершить свою обработку, прежде чем сможет начаться следующий.

```
    final List<String> list = List.of("one", "two", "three");

    list.stream()
            .filter(s -> {
                System.out.println("filter: " + s);
                return s.length() <= 3;
            })
            .map(s1 -> {
                System.out.println("map: " + s1);
                return s1.toUpperCase();
            })
            .sorted()
            .forEach(x -> {
                System.out.println("forEach: " + x);
            });
}
```

Порядок вывода в forEach() методе меняется в зависимости от результата сортировки. sorted() операция создаёт «точку синхронизации», в которой все элементы должны быть обработаны в первую очередь, прежде чем конвейер сможет завершить работу.

```
filter: one
map: one
filter: two
map: two
filter: three
forEach: ONE
forEach: TWO

Process finished with exit code 0
```

Spliterator

В основе всех коллекций в Java лежит интерфейс Iterator, позволяющий последовательно перебирать элементы.

Для работы с потоками данных в Stream API используется более мощный механизм — Spliterator.

Его ключевая особенность — возможность разделять данные для независимой обработки несколькими потоками.

Интерфейс Spliterator определяют четыре ключевых метода:

- **long estimateSize()** возвращает оценку количества элементов, оставшихся в коллекции.
- **boolean tryAdvance(Consumer<? super T> action)** обрабатывает следующий элемент коллекции, применяя к нему функцию. Если элемент существует, функция возвращает true (иначе false).
- **int characteristics()** возвращает набор битовых флагов, описывающих характеристики текущего Spliterator. Эти флаги помогают оптимизировать обработку потока на основе свойств данных, таких как порядок или уникальность.
- **Spliterator<T> trySplit()** метод пытается разделить текущий массив Spliterator на две части. Он возвращает новый массив Spliterator, который будет содержать часть данных, в то время как исходный Spliterator массив сокращается для обработки оставшейся части. Если разделение невозможно, он возвращает null.

IThub

```
AtomicInteger i = new AtomicInteger(0);

List<String> words = Stream.generate(() -> {
            i.incrementAndGet();
            return  "word " + i;
        }).limit(10)
        .collect(Collectors.toList());

Spliterator<String> spliterator = words.spliterator();
```

```
while (spliterator.tryAdvance(System.out::println));
```

tryAdvance обрабатывает следующий элемент коллекции, применяя к нему функцию

```
word 1
word 2
word 3
word 4
word 5
word 6
word 7
word 8
word 9
word 10

Process finished with exit code 0
```

IThub

trySplit() метод пытается разделить текущий массив Spliterator на две части

```
Spliterator<String> spliterator = words.spliterator();
Spliterator<String> split2 = spliterator.trySplit();

while (spliterator.tryAdvance(System.out::println));
System.out.println("-----");
while (split2.tryAdvance(System.out::println));
```

```
word 6
word 7
word 8
word 9
word 10
-----
word 1
word 2
word 3
word 4
word 5

Process finished with exit
code 0’
```

IThub

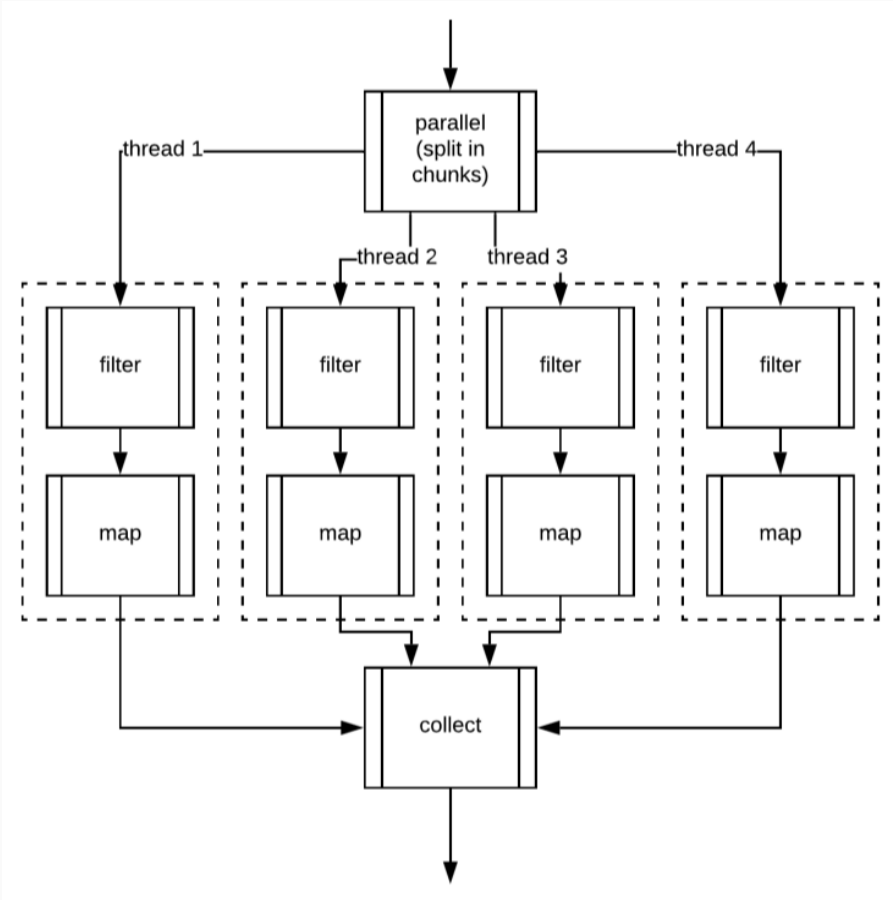
Параллельное выполнение

API Stream предоставляет возможность параллельной обработки данных, что может значительно повысить производительность.

Используются методы parallelStream() или parallel().

Для разделения коллекций на части, которые можно обрабатывать параллельно, Java использует Spliterator.

Вместо одного набора операций создаются несколько их копий, каждая из которых выполняется над своим сегментом данных, а после завершения результаты объединяются в итоговый вывод.



IThub

```
@Test
@Ignore
public void parallelStreamTest() {
    AtomicInteger i = new AtomicInteger(0);

    List<Integer> numbers = Stream.generate((i::incrementAndGet))
            .limit(100000000)
            .collect(Collectors.toList());

    long startTime = System.nanoTime();
    List<Integer> evenNumbers = numbers.stream()
            .filter(n-> n % 2 == 0)
            .sorted((n1, n2) -> n2.compareTo(n1))
            .collect(Collectors.toList());
    long endTime = System.nanoTime();
    long parallelTime = endTime - startTime;
    System.out.println("WITHOUT parallel execution Time = " + parallelTime/1000000 + " ms");

    startTime = System.nanoTime();
    List<Integer> evenNumbers2 = numbers.stream().parallel()
            .filter(n-> n % 2 == 0)
            .sorted((n1, n2) -> n2.compareTo(n1))
            .collect(Collectors.toList());
    endTime = System.nanoTime();
    parallelTime = endTime - startTime;
    System.out.println("WITH parallel execution Time = " + parallelTime/1000000 + " ms");
}
```

```
WITHOUT parallel execution Time = 4279 ms
WITH parallel execution Time = 5752 ms

Process finished with exit code 0
```

Ни в коем случае не надо использовать parallelStream всегда! Его надо использовать исключительно редко, и у вас должен быть хороший повод для этого

Когда использовать параллельный поток в Java

Параллельная обработка может быть полезна для полного использования нескольких ядер. Однако необходимо также учитывать накладные расходы на управление несколькими потоками, локальность памяти (повторно использовать данные, расположенным рядом с недавно использованными адресами памяти), разделение исходного кода и слияние результатов.

Иногда **накладные расходы** на управление потоками, источниками и результатами **обходятся дороже, чем выполнение самой работы**.

```
List<Integer> arrayListOfNumbers = new ArrayList<>();
List<Integer> linkedListOfNumbers = new LinkedList<>();
IntStream.rangeClosed(1, 1_000_000).forEach(i -> {
    arrayListOfNumbers.add(i);
    linkedListOfNumbers.add(i);
});

long startTime = System.nanoTime();
arrayListOfNumbers.stream().reduce(0, Integer::sum); // суммируем все элементы
long endTime = System.nanoTime();
long execTime = endTime - startTime;
System.out.println("ArrayList reduce stream() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
arrayListOfNumbers.parallelStream().reduce(0, Integer::sum);
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("ArrayList reduce parallelStream() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
linkedListOfNumbers.stream().reduce(0, Integer::sum);
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("LinkedList reduce stream() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
linkedListOfNumbers.parallelStream().reduce(0, Integer::sum);
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("LinkedList reduce parallelStream() = " + execTime/1000000 + " ms");
```

Массивы можно легко и равномерно разбить , в то время как LinkedList не обладает ни одним из этих свойств. TreeMap и HashSet разбиваются лучше, чем LinkedList, но не так хорошо, как массивы.

```
ArrayList reduce stream() = 150 ms
ArrayList reduce parallelStream() = 43 ms

LinkedList reduce stream() = 24 ms
LinkedList reduce parallelStream() = 31 ms

Process finished with exit code 0
```

```
long startTime = System.nanoTime();
arrayListOfNumbers.stream().reduce(0, Integer::sum);
long endTime = System.nanoTime();
long execTime = endTime - startTime;
System.out.println("ArrayList reduce stream() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
arrayListOfNumbers.stream().parallel().reduce(0, Integer::sum);
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("ArrayList reduce parallel() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
arrayListOfNumbers.stream().collect(Collectors.toSet());
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("ArrayList collect stream() = " + execTime/1000000 + " ms");

startTime = System.nanoTime();
arrayListOfNumbers.stream().parallel().collect(Collectors.toSet());
endTime = System.nanoTime();
execTime = endTime - startTime;
System.out.println("ArrayList collect parallel() = " + execTime/1000000 + " ms");
```

```
ArrayList reduce stream() = 70 ms
ArrayList reduce parallel() = 34 ms

ArrayList collect stream() = 104 ms
ArrayList collect parallel() = 285 ms

Process finished with exit code 0
```

Преобразование последовательного потока в параллельный дает выигрыш в производительности только для операции суммирования.

Операция слияния действительно дешева для некоторых операций, таких как вычитание или сложение, но операции слияния, такие как группировка в наборы (Set) или карты (Map), могут быть **довольно дорогими**.

IThub

Преимущества потоков

Более компактный код

- Уменьшает объем кода, необходимого для обработки коллекции.

Меньше промежуточных переменных

- Промежуточные переменные могут быть причиной совершения ошибок.
- Чем их меньше — тем проще избежать неожиданных ошибок.

Интуитивно понятный код

- Позволяет описывать, что нужно сделать с данными, а не расписывать каждый шаг вручную.

Задание

IThub

Order
Long id
LocalDateTime createdAt
OrderStatus status
Customer customer
List<OrderItem> items
Long totalPrice

OrderItem
Product product
Integer quantity
Integer discount

Customer
Long id
String name
String email
Address address

Product
Long id
String name
Long price
ProductCategory productCategory

Мы уже делали модель интернет-магазина с заказами в теме с Jakson

Address
String country
String city
String street
Long houseNumber

OrderStatus	CREATED, PAID, SHIPPED, CANCELLED.
ProductCategory	ELECTRONICS, BOOKS, CLOTHING, HOME

Задание 1

Фильтрация заказов List<Order> (предварительно создайте список и заполните данными)

Найдите все заказы со статусом PAID, стоимость которых превышает 10 000 рублей

- используйте filter() дважды
- сохраните результат в List<Order>

IThub

Задание 2

Рассчитайте общую выручку магазина по всем заказам со статусом PAID или SHIPPED.

- сначала отфильтруйте нужные заказы.
- используйте reduce() для суммирования totalPrice
- начальное значение = 0
- не учитывайте отменённые и ещё не оплаченные заказы

Решение задание 1

```
List<Order> filteredOrders = orders.stream()
        .filter(order -> order.getStatus() == OrderStatus.PAID)
        .filter(order -> order.getTotalPrice() > 10_000L)
        .filter(order -> !order.getItems().isEmpty())
        .toList();

filteredOrders.forEach(order -> System.out.println(
        order.getId() + " | "
        + order.getCustomer().getName() + " | "
        + order.getTotalPrice()
));
```

Решение задание 2

```
Long revenue = orders.stream()
        .filter(order ->
                order.getStatus() == OrderStatus.PAID ||
                order.getStatus() == OrderStatus.SHIPPED
        )
        .map(Order::getTotalPrice)
        .reduce(0L, Long::summ);

System.out.println("Общая выручка: " + revenue);
```

IThub

Задание 3

Сгруппируйте заказы по OrderStatus в Map<OrderStatus, List<Order>> groupedOrders

Задание 4

Разделите все заказы на две группы:

- заказы стоимостью от 10 000 рублей включительно;
- заказы дешевле 10 000 рублей.

Результат в Map<Boolean, List<Order>> partitionedOrders;

Решение задание 3

```
Map<OrderStatus, List<Order>> groupedOrders =
        orders.stream()
                .collect(Collectors.groupingBy(Order::getStatus));
```

Решение задание 4

```
Map<Boolean, List<Order>> partitionedOrders =
        orders.stream()
                .collect(Collectors.partitioningBy(
                        order -> order.getTotalPrice() >= 10_000L
                ));
```